C PROGRAMMING • STUDY GUIDE

Loop Control Structures in C

A Complete Exam-Oriented Reference

- ✓ Concept of loops & iteration in C
- ✓ Top-tested vs Bottom-tested loops
- ✓ for, while and do-while statements
- ✓ Nested loops & loop variants
- ✓ Increment / decrement operators
- ✓ break and continue statements
- ✓ Comparative analysis & solved programs
- ✓ One-page quick revision notes

Designed for University Examinations • Competitive Examinations • Teacher Recruitment Tests (TET/TGT/PGT/NET)
Includes flowcharts, syntax boxes, comparison tables and solved examples with output.

Table of Contents

1. Introduction to Loop Control Structures **03**

2. Concept of Loops **04**

3. Top-Tested and Bottom-Tested Loops **05**

4. The *for* Loop Statement **07**

5. Nested *for* Loops **09**

6. *for* Loop Variants **11**

7. The *while* Loop Statement **13**

8. Increment and Decrement Operators **15**

9. *break* and *continue* Statements **17**

10. The *do-while* Loop **19**

11. Comparison of Loop Structures **21**

12. Programming Examples **23**

13. Quick Revision Section **26**

NOTE: Page numbers correspond to the printed layout and may shift slightly depending on the rendering device.

1. Introduction to Loop Control Structures

Definition: A **loop** is a control structure in C that allows a block of statements (called the **loop body**) to be executed **repeatedly** as long as a specified condition remains true. Loops are one of the fundamental control structures in programming, used to repeat a set of instructions efficiently.

NOTE: Every program is built using three fundamental control structures:

- **Sequence** (or Sequential Control) — steps executed one after another, in order
- **Selection** (or Conditional Control / Decision Control) — a choice made using `if`, `else`, `switch`
- **Iteration** (or Repetition / Loop Control) — a block of code repeated using `for`, `while`, `do-while`

Loops fall under the third category — **Iteration**.

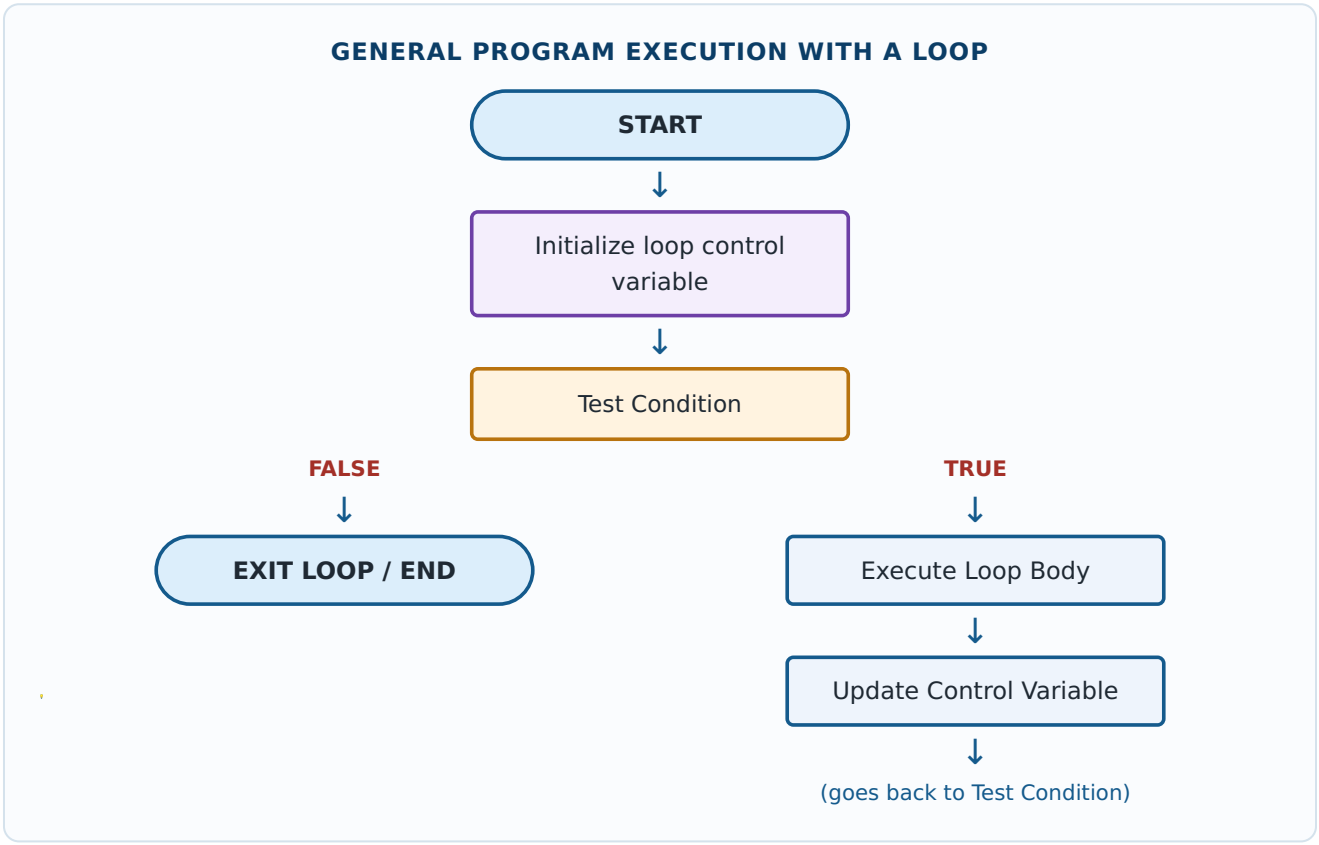
1.1 Why Do We Need Loops?

- To avoid writing the same statement multiple times (code reuse).
- To process repetitive tasks such as reading arrays, printing patterns, or summing a series of numbers.
- To reduce program length, improve readability, and reduce chances of error.
- To allow a program to work with a variable/unknown number of repetitions decided at run time.

1.2 Types of Loops Supported by C

C provides **three** types of loop control statements, classified on the basis of **when** the condition is tested:

Loop Type	Category	Condition Checked
`for` loop	Entry-controlled (Top-tested)	Before the loop body executes
`while` loop	Entry-controlled (Top-tested)	Before the loop body executes
`do-while` loop	Exit-controlled (Bottom-tested)	After the loop body executes



EXAM TIP: Remember the acronym **I-C-U** for every loop: **I**nitialization, **C**ondition, **U**pdate — every loop, regardless of type, needs all three components to function correctly and terminate.

2. Concept of Loops

2.1 The Iteration Process

Iteration means repeating a set of instructions until a defined condition is satisfied. Every iterative process in C is built from three essential components:

Component	Purpose	Example
Initialization	Sets the starting value of the loop control (counter) variable. Executed only **once**.	`int i = 1;`
Condition (Test Expression)	A Boolean expression checked before/after each iteration; loop continues while it is **true**.	`i <= 10`
Update (Increment/ Decrement)	Modifies the loop control variable after each iteration, eventually making the condition false.	`i++` or `i--`

⚠ **COMMON ERROR:** If the update step is missing or never makes the condition false, the loop becomes an **infinite loop** — one of the most common logical errors made by beginners.

2.2 Advantages of Using Loops

- **Code reusability** — the same block executes multiple times without duplication.
- **Reduced program size** and improved maintainability.
- **Efficiency** in handling arrays, strings, matrices, and files.
- Enables **dynamic repetition** — the number of iterations can depend on user input or run-time data.
- Forms the basis of algorithms such as searching, sorting, and numerical computation.

2.3 Applications of Loops

- Traversing arrays and strings
- Summation and averaging of a series
- Generating patterns and number series (Fibonacci, factorial)
- Searching and sorting algorithms
- Menu-driven programs (repeat until user exits)
- Matrix operations using nested loops

Chapter Summary

- A loop repeats a block of code based on a condition.
- C supports three loops: `for`, `while`, `do-while`.
- Every loop needs Initialization → Condition → Update.
- Loops improve reusability, reduce code size, and enable dynamic repetition.

3. Top-Tested and Bottom-Tested Loops

3.1 Top-Tested (Entry-Controlled) Loop

Definition: A top-tested loop is one in which the test condition is evaluated **before** the loop body is executed. If the condition is false at the very first check, the body may execute **zero times**. C examples: `for`, `while`.

CHARACTERISTICS

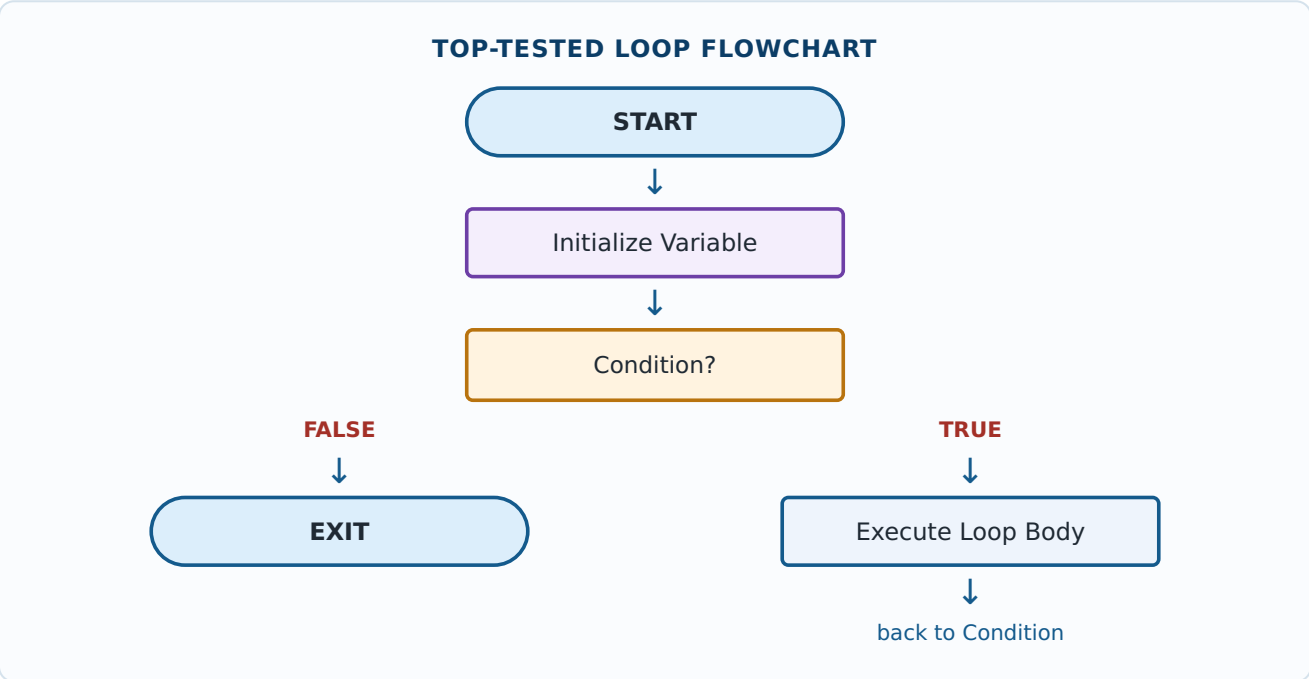
- Condition is checked **before** entering the loop body.
- Body may execute **zero** or more times.
- Also called **pre-tested** or **entry-controlled** loop.
- Used when the number of iterations depends on a condition checked upfront.

EXAMPLE

```
int i = 10;
while (i < 5) {
    printf("%d", i);   /* never executes since 10 < 5 is false */
}
```

OUTPUT

```
(no output — body never runs)
```



3.2 Bottom-Tested (Exit-Controlled) Loop

Definition: A bottom-tested loop is one in which the test condition is evaluated **after** the loop body is executed. The body always executes **at least once**, regardless of the condition. C example: `do-while`.

CHARACTERISTICS

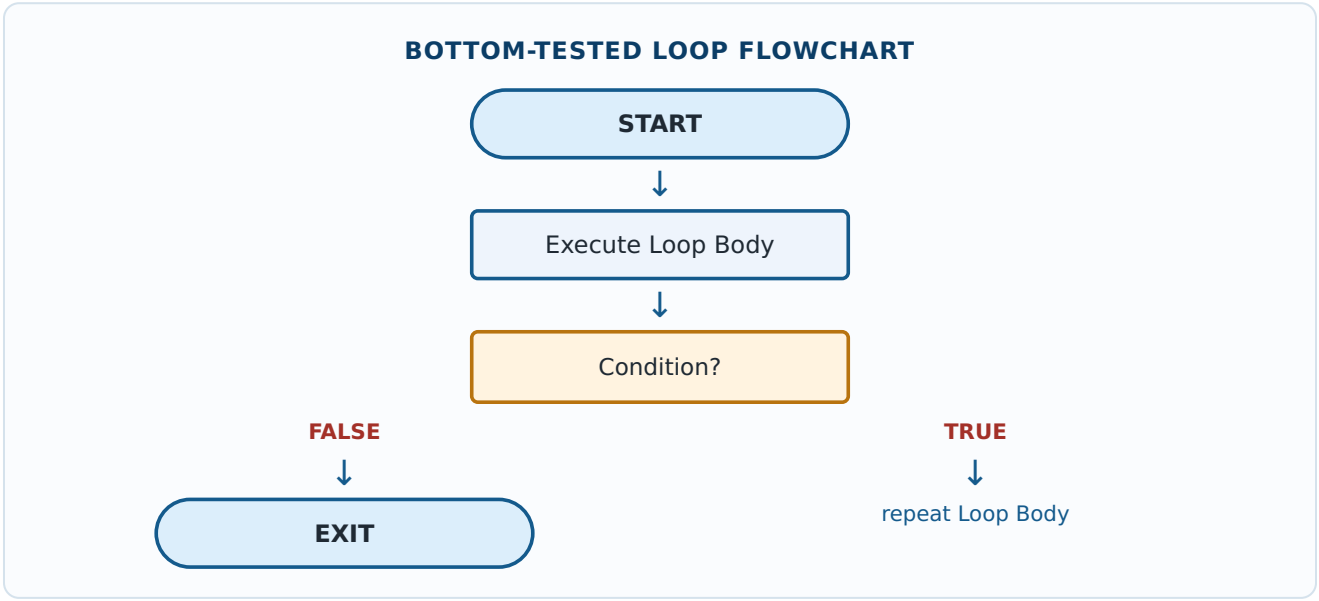
- Condition is checked **after** executing the loop body.
- Body executes **at least once**, no matter what.
- Also called **post-tested** or **exit-controlled** loop.
- Used when the body must run at least once, e.g. menu-driven programs.

EXAMPLE

```
int i = 10;
do {
    printf("%d", i);   /* executes once even though condition is false */
} while (i < 5);
```

OUTPUT

```
10
```



3.3 Differences Between Top-Tested and Bottom-Tested Loops

Feature	Top-Tested Loop	Bottom-Tested Loop
Condition checked	Before executing the body	After executing the body
Minimum executions	0 (may not execute at all)	1 (executes at least once)
	`while(cond)` appears before body	`while(cond);` appears after body

Keyword position		
Also called	Pre-tested / Entry-controlled	Post-tested / Exit-controlled
C examples	`for`, `while`	`do-while`
Typical use	When number of iterations depends on a condition checked upfront	When the body must run at least once (e.g. menu-driven programs)

EXAM TIP: A classic exam question: "Which loop in C always executes at least once?" — Answer: **do-while**, because it is bottom-tested / exit-controlled.

4. The *for* Loop Statement

Definition: The `for` loop is an entry-controlled loop that combines initialization, condition-testing, and updating into a single line, making it ideal when the number of iterations is known in advance.

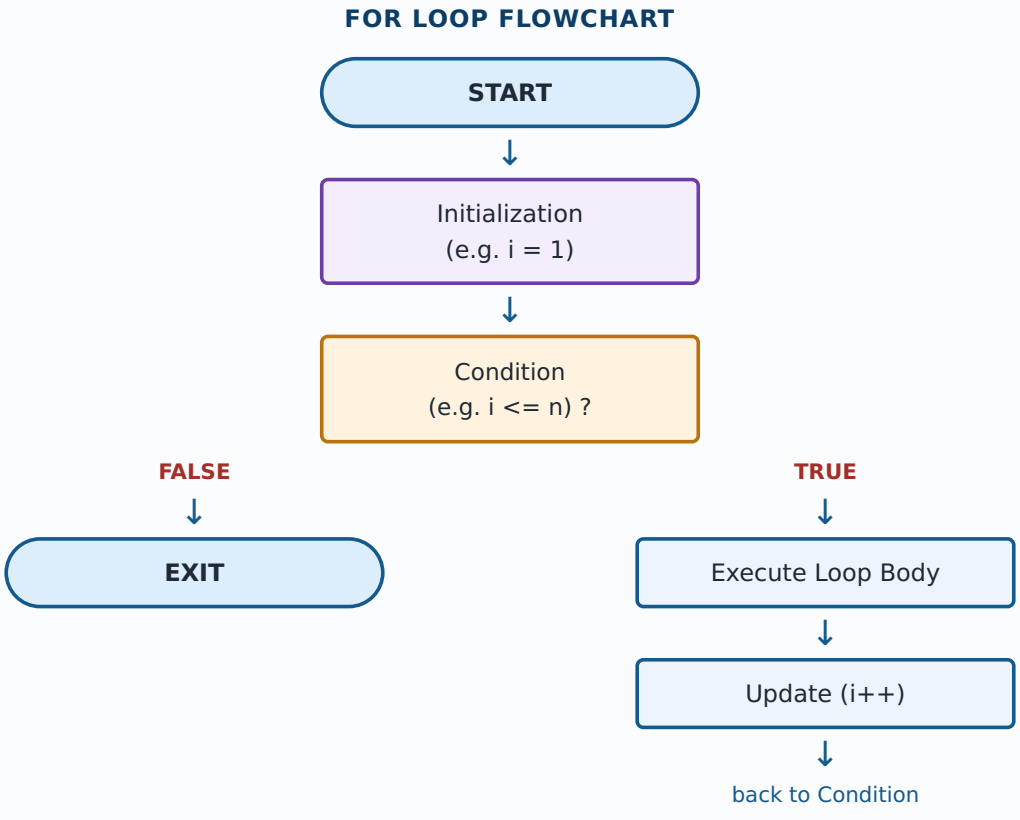
4.1 Syntax

GENERAL FORM

```
for (initialization; condition; update) {
    // loop body / statements
}
```

4.2 Working Mechanism

1. **Initialization** executes once, before the loop starts.
2. **Condition** is evaluated; if **true**, control enters the loop body; if **false**, control exits the loop.
3. The **loop body** statements execute.
4. The **update** expression executes (increment/decrement).
5. Control returns to Step 2, and the cycle repeats until the condition becomes false.



4.3 Example with Output

```c
#include <stdio.h>
int main() {
    int i;
    for (i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

4.4 Advantages and Limitations

Advantages	Limitations
Compact — initialization, condition and update in one line	Less readable for very complex update logic
Ideal when the number of iterations is known/fixed	Loop variable scope can leak if not declared inside the `for` (in older C standards)
Easy to modify step size (e.g. `i += 2`)	Not as intuitive as `while` for condition-driven repetition

⚠ **COMMON ERROR:** Placing a semicolon immediately after the `for(...)` header, e.g. `for(i=1;i<=5;i++);` creates an **empty loop body** — a very common exam trick and beginner bug.

5. Nested *for* Loops

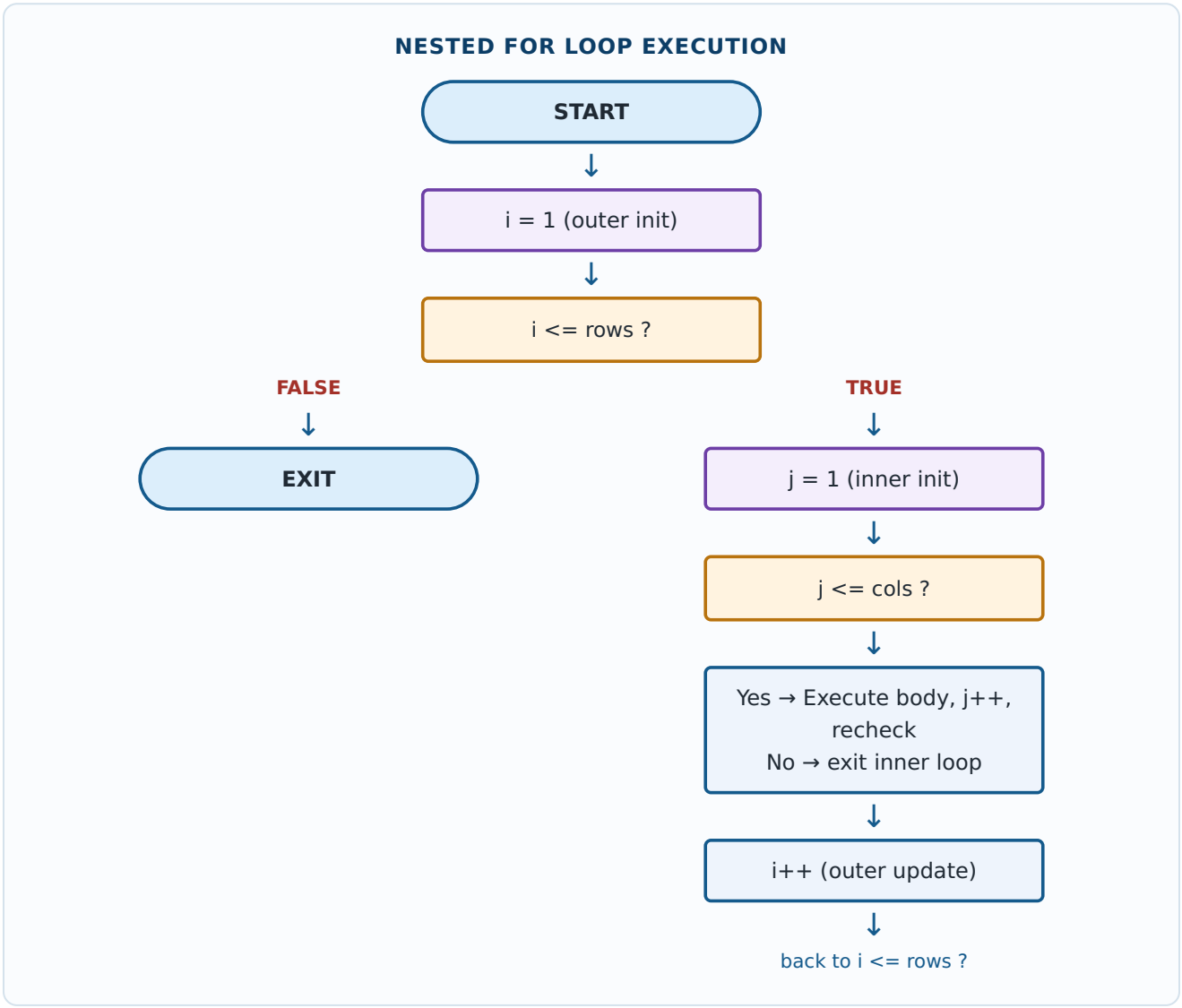
Definition: A **nested loop** is a loop placed inside the body of another loop. The loop that contains another is called the **outer loop**; the loop inside is the **inner loop**. For every single iteration of the outer loop, the inner loop completes **all** of its iterations.

5.1 Syntax

```
for (i = 1; i <= rows; i++) {          // outer loop
    for (j = 1; j <= cols; j++) {      // inner loop
        // statements — executes rows × cols times
    }
}
```

5.2 Execution Process

- Outer loop initializes and checks its condition → enters body.
- Inner loop runs to **completion** (all its iterations) for that single pass of the outer loop.
- Control returns to the outer loop, which updates and re-checks its own condition.
- The process repeats until the outer loop's condition becomes false.



5.3 Pattern Program Example

```c
#include <stdio.h>
int main() {
    int i, j;
    for (i = 1; i <= 4; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
*
* *
* * *
* * * *
```

5.4 Another Example — Multiplication Grid

```
for (i = 1; i <= 3; i++) {
    for (j = 1; j <= 3; j++)
        printf("%d\t", i * j);
    printf("\n");
}
```

OUTPUT

```
1  2  3
2  4  6
3  6  9
```

EXAM TIP: Total number of inner-loop iterations for a nested loop = (iterations of outer loop) × (iterations of inner loop) — a frequent numerical/aptitude question.

6. *for* Loop Variants

C's `for` loop syntax is flexible — each of the three sections (initialization, condition, update) can be modified, combined, or even omitted.

6.1 Multiple Initialization Expressions

Use the comma operator to initialize more than one variable.

```
for (i = 0, j = 10; i < 5; i++) {
    printf("%d %d\n", i, j);
}
```

OUTPUT

```
0 10
1 10
2 10
3 10
4 10
```

6.2 Multiple Condition Expressions

The comma operator can also be used in the condition section to evaluate more than one expression. In C, the comma operator evaluates **all** expressions but the loop continues based on the value of only the **rightmost (last)** expression.

```
for (i = 0, j = 10; i < 5, j > 5; i++, j--) {
    printf("i=%d j=%d\n", i, j);
}
```

OUTPUT

```
i=0 j=10
i=1 j=9
i=2 j=8
i=3 j=7
i=4 j=6
/* loop stops once j > 5 becomes false — the leftmost i < 5 is evaluated but ignored
for the test */
```

⚠ **COMMON ERROR:** In `i < 5, j > 5`, only `j > 5` (the last expression) decides whether the loop continues. This is a common confusion point — the comma operator does **not** combine conditions like && does. To test multiple conditions together, use logical operators instead: `for (i = 0, j = 10; i < 5 && j > 5; i++, j--)`.

6.3 Multiple Update Expressions

The comma operator can also combine more than one update expression, allowing two (or more) variables to be modified after every iteration.

```
for (i = 0, j = 10; i < j; i++, j--) {
    printf("i=%d j=%d\n", i, j);
}
```

OUTPUT

```
i=0 j=10
i=1 j=9
i=2 j=8
i=3 j=7
i=4 j=6
```

NOTE: All update expressions separated by commas execute on every iteration, in left-to-right order — here `i` increases while `j` decreases simultaneously, so the loop ends as soon as `i < j` becomes false.

6.4 Infinite *for* Loop

If any (or all) of the three expressions is omitted while keeping the semicolons, and no `break` is used, the loop runs forever.

```
for ( ; ; ) {
    printf("This runs forever\n");   /* needs a break to terminate */
}
```

⚠ **COMMON ERROR:** An intentional infinite loop is common in embedded systems and server programs, but must contain a controlled exit (usually via `break` or a `return`) to avoid a program hang.

6.5 Omitted Expressions

Variant	Example	Remarks
Omit initialization	`for (; i <= 5; i++)`	Variable must be initialized before the loop
Omit condition	`for (i = 0; ; i++)`	Condition treated as always-true; needs internal `break`
Omit update	`for (i = 0; i < 5;)`	Update must be done manually inside the body
Omit all three	`for (;;)`	Classic infinite loop notation
Loop body with no statements	`for (i = 0; i < 5; i++);`	Null/empty statement loop (often a bug)

6.6 Practical Example — Step Value Other Than 1

```
for (i = 0; i <= 20; i += 5) {
    printf("%d ", i);
}
```

OUTPUT

```
0 5 10 15 20
```

EXAM TIP: Exam favourite: identifying the output of a `for` loop with a comma operator or an omitted section — always trace initialization, condition, body, and update carefully step-by-step.

7. The *while* Loop Statement

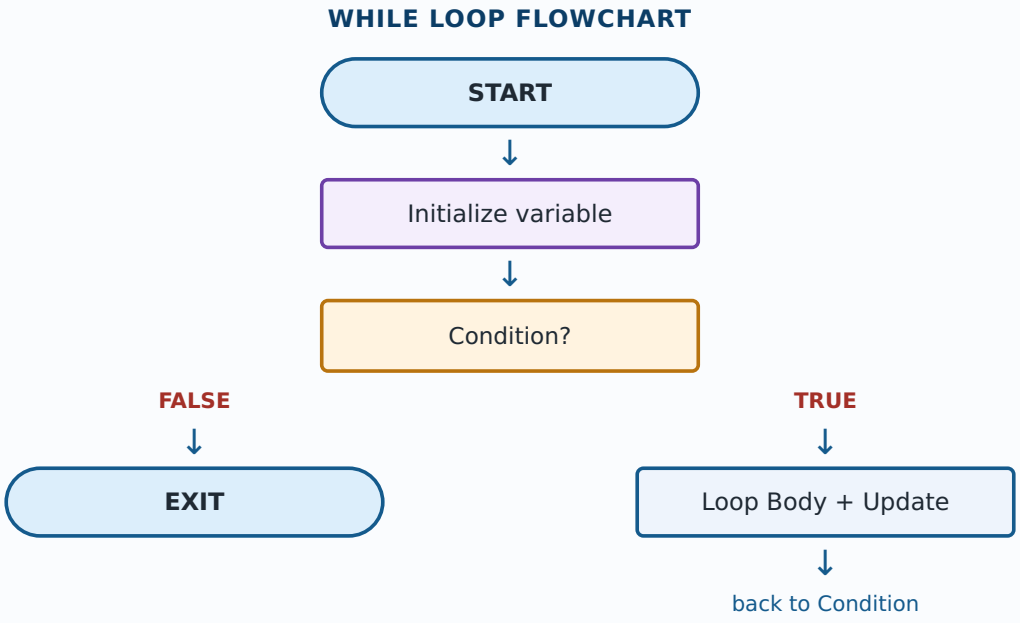
Definition: The `while` loop is an entry-controlled (top-tested) loop that repeats a block of statements **as long as** a given condition remains true. It is best suited when the number of iterations is **not known in advance**.

7.1 Syntax

```
initialization;
while (condition) {
    // loop body
    update;
}
```

7.2 Working / Top-Tested Behaviour

- The condition is tested **first**; if false initially, the body never executes (0 iterations possible).
- If true, the body executes once, then the condition is re-tested.
- The cycle continues until the condition evaluates to false.



7.3 Simple *while* Loop — Example

```
#include <stdio.h>
int main() {
    int i = 1;
    while (i <= 5) {
        printf("%d ", i);
        i++;
    }
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

7.4 Nested *while* Loop

```
int i = 1, j;
while (i <= 3) {
    j = 1;
    while (j <= i) {
        printf("%d ", j);
        j++;
    }
    printf("\n");
    i++;
}
```

OUTPUT

```
1
1 2
1 2 3
```

NOTE: Just like nested `for` loops, the **inner while loop completes all of its iterations** for every single pass of the outer loop. Notice that the inner loop variable `j` must be **re-initialized** (`j = 1`) each time before the inner loop starts — otherwise it would not restart correctly on the next outer pass.

⚠ **COMMON ERROR:** Forgetting to update the loop control variable inside a `while` loop body (unlike `for`, there is no built-in update section) is the single most common cause of an unintended infinite loop.

8. Increment and Decrement Operators

Definition: C provides unary **increment (++)** and **decrement (--)** operators that add or subtract 1 from a variable. They are extensively used to update loop control variables.

8.1 Pre-Increment and Post-Increment

Operator	Form	Behaviour
Pre-increment	`++i`	Value is incremented **first**, then used in the expression
Post-increment	`i++`	Current value is used in the expression **first**, then incremented
Pre-decrement	`--i`	Value is decremented **first**, then used
Post-decrement	`i--`	Current value is used **first**, then decremented

8.2 Working Mechanism — Example

```
int i = 5, a, b;
a = ++i;   // i becomes 6 first, then a = 6
printf("i=%d a=%d\n", i, a);

int j = 5, c;
c = j++;   // c = 5 (old value), then j becomes 6
printf("j=%d c=%d\n", j, c);
```

OUTPUT

```
i=6 a=6
j=6 c=5
```

8.3 Comparison Table

Aspect	Pre-increment / Pre-decrement	Post-increment / Post-decrement
Order of operation	Update happens before use	Use happens before update
Symbol placement	Before variable (`++i`, `--i`)	After variable (`i++`, `i--`)
Value returned in expression	New (updated) value	Old (original) value
Common use in loops	Same net effect as post in standalone statements like `i++;`	Most common style: `for(i=0;i<n;i++)`

8.4 Use in Loop Updates

```
for (i = 0; i < 5; i++)     // post-increment — most common
for (i = 5; i > 0; --i)     // pre-decrement — equally valid here
```

EXAM TIP: When the increment/decrement operator is used as a **standalone statement** (e.g. `i++;` on its own line), pre- and post- forms behave **identically**. The difference only matters when the expression's **value** is used, e.g. in an assignment like `a = i++;`.

9. *break* and *continue* Statements

9.1 The *break* Statement

Definition: The `break` statement immediately terminates the **innermost** enclosing loop (or `switch`) and transfers control to the first statement **after** the loop.

SYNTAX

```
break;    // used inside loops or switch
```

WORKING

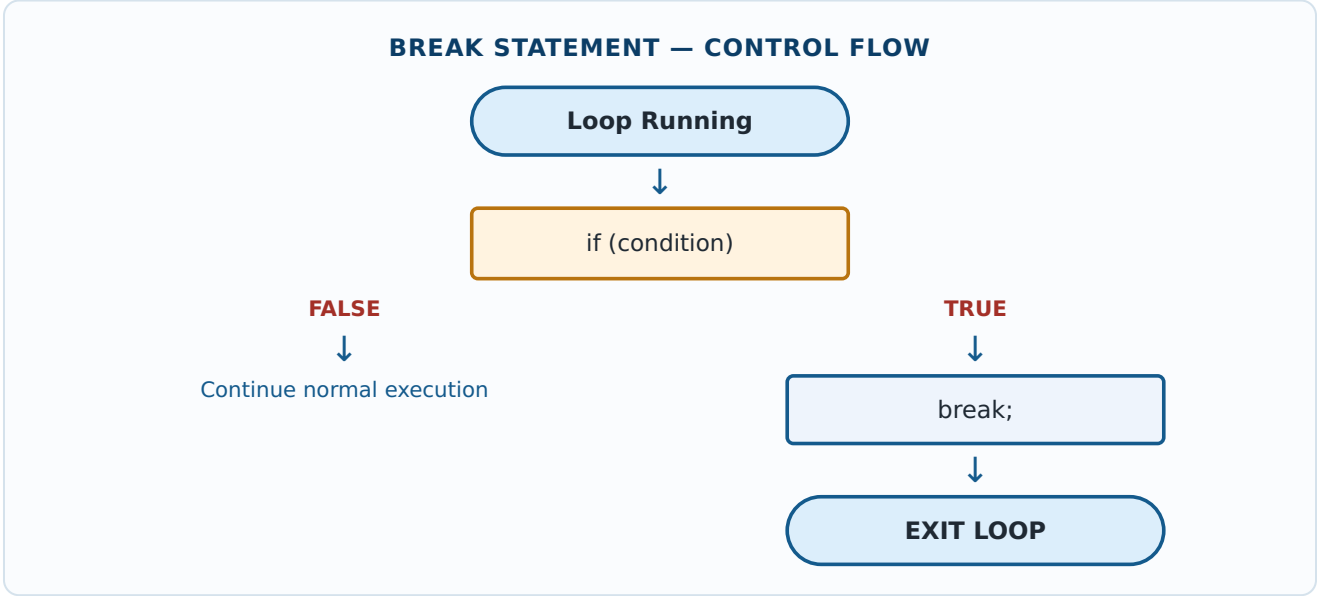
- When `break` is encountered, the loop stops **immediately** — no further iterations occur.
- Control jumps straight to the statement right after the closing brace of the loop.
- Commonly used to stop a search as soon as a match is found (e.g. prime checking).

EXAMPLE WITH OUTPUT

```
for (i = 1; i <= 10; i++) {
    if (i == 5)
        break;
    printf("%d ", i);
}
```

OUTPUT

```
1 2 3 4
```



9.2 The *continue* Statement

Definition: The `continue` statement skips the **remaining statements** in the current iteration and jumps directly to the **update/condition check** of the loop, continuing with the **next** iteration.

SYNTAX

```
continue;    // used inside loops only
```

WORKING

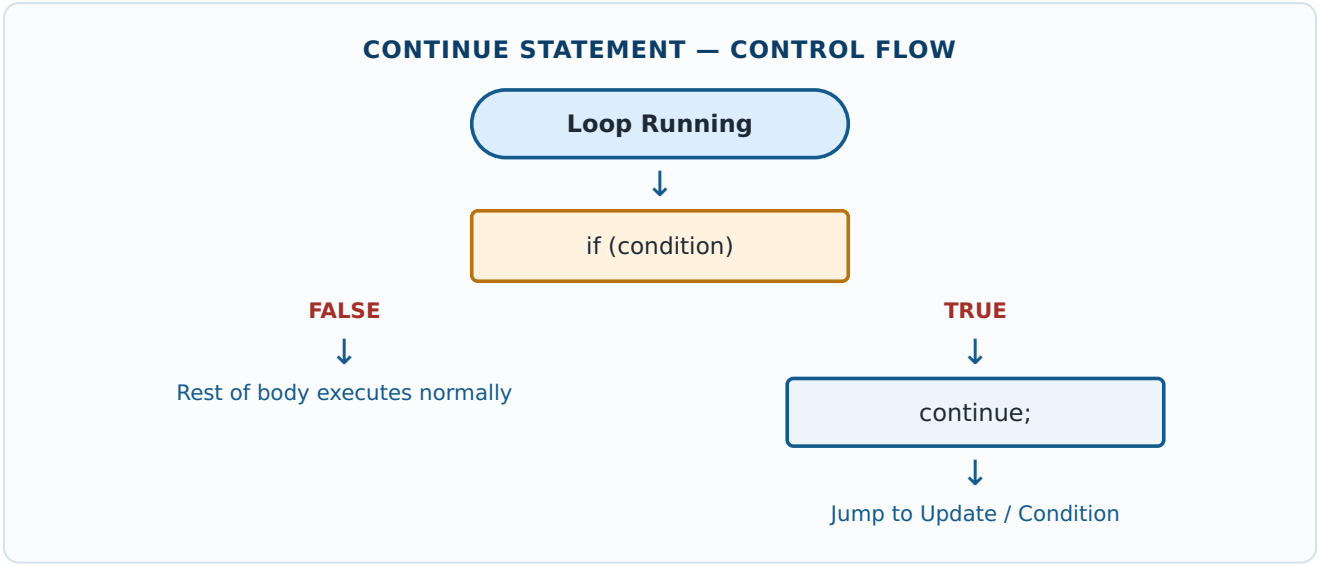
- When `continue` is encountered, the rest of the loop body for that iteration is **skipped**.
- In a `for` loop, control jumps to the **update** expression, then re-checks the condition.
- In a `while` / `do-while` loop, control jumps directly to the **condition test**.
- Commonly used to skip unwanted values while still continuing the loop (e.g. skipping multiples of a number).

EXAMPLE WITH OUTPUT

```
for (i = 1; i <= 5; i++) {
    if (i == 3)
        continue;
    printf("%d ", i);
}
```

OUTPUT

```
1 2 4 5
```



9.3 break vs continue — Working Process

Statement	Effect on Loop	Where control goes
`break`	Loop terminates completely	First statement after the loop

`continue`	Current iteration ends early	`for`: to update step; `while`/`do-while`: to condition test

9.4 Use in Nested Loops

NOTE: `break` and `continue` affect only the **innermost** loop in which they are written. To exit multiple nested loops, additional techniques such as flags or `goto` are required (rarely tested at introductory level but a good conceptual point).

EXAM TIP: Common exam trap: distinguishing `break` (loop stops entirely) from `continue` (loop skips only the current pass). Trace the printed sequence carefully in MCQs.

10. The *do-while* Loop

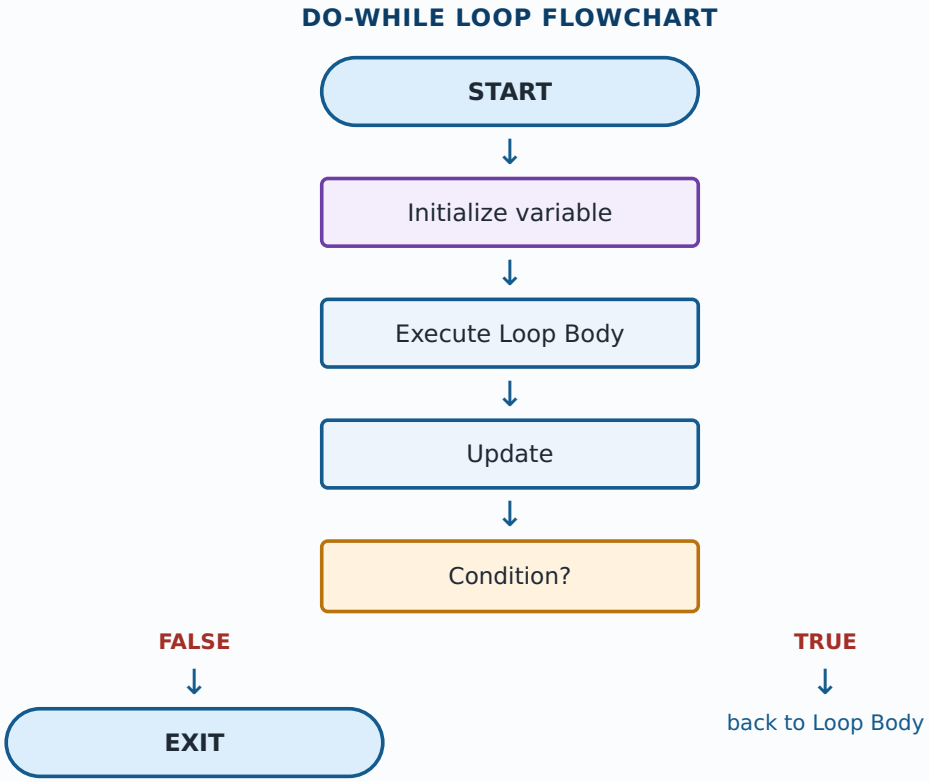
Definition: The `do-while` loop is an exit-controlled (bottom-tested) loop in which the loop body executes **first**, and the condition is checked **afterward**. It guarantees the body runs **at least once**.

10.1 Syntax

```
initialization;
do {
    // loop body
    update;
} while (condition);   // note the semicolon — mandatory
```

10.2 Bottom-Tested Behaviour & Execution Process

1. Loop body executes **unconditionally** the first time.
2. After execution, the condition is evaluated.
3. If true, control jumps back to repeat the body; if false, the loop terminates.



10.3 Example with Output

```c
#include <stdio.h>
int main() {
    int i = 1;
    do {
        printf("%d ", i);
        i++;
    } while (i <= 5);
    return 0;
}
```

OUTPUT

```
1 2 3 4 5
```

10.4 Example — Body Executes Once Despite False Condition

```c
int i = 20;
do {
    printf("Value: %d\n", i);
} while (i < 10);
```

OUTPUT

```
Value: 20
```

10.5 Advantages and Limitations

Advantages	Limitations
Guarantees at least one execution — ideal for menu-driven programs	Can accidentally execute unwanted code once if condition is meant to guard entry
Useful for input-validation loops (prompt at least once)	Slightly less intuitive to read than `while` for beginners
Clear separation of "action" then "check"	Requires a trailing semicolon — easy to forget, causing compile errors

⚠ **COMMON ERROR:** Missing the semicolon after `while(condition)` in a `do-while` loop is a **syntax error** — unlike the `for` and `while` loops, the semicolon here is compulsory.

11. Comparison of Loop Structures

11.1 *for* vs *while*

Basis	`for` Loop	`while` Loop
Structure	Init, condition, update in one line	Init outside; condition in header; update inside body
Best suited when	Number of iterations is known	Number of iterations is unknown / condition-driven
Readability for counters	**More compact and readable**	Requires separate lines for init/update
Testing	Entry-controlled (top-tested)	Entry-controlled (top-tested)

11.2 *while* vs *do-while*

Basis	`while` Loop	`do-while` Loop
Condition check	Before the body (top-tested)	After the body (bottom-tested)
Minimum executions	0	**Always ≥ 1**
Terminating symbol	No semicolon after `while(cond)`	Semicolon mandatory after `while(cond);`
Typical use case	General repetition with upfront check	Menus, input validation, "try at least once" logic

11.3 *for* vs *do-while*

Basis	`for` Loop	`do-while` Loop
Testing type	Entry-controlled	Exit-controlled
Structure compactness	**All three parts combined**	Parts written separately
Guaranteed execution	Not guaranteed	**Guaranteed at least once**
Common usage	Counting, arrays, fixed iteration	Menu-driven / validation programs

11.4 Consolidated Comparison

Feature	`for`	`while`	`do-while`
Control type	Entry-controlled	Entry-controlled	Exit-controlled
Minimum iterations	0	0	1

Init/Condition/Update location	All in header	Init outside, update inside body	Init outside, update inside body
Semicolon after condition	No	No	Yes
Best for	Known iteration count	Unknown iteration count	At-least-once execution

11.5 Selection Guidelines

- Use **`for`** when the number of repetitions is fixed or countable (e.g. traversing an array of known size).
- Use **`while`** when repetition depends on a condition whose outcome isn't known beforehand (e.g. reading input until EOF).
- Use **`do-while`** when the task must be performed **at least once** before checking whether to repeat (e.g. displaying a menu).

Chapter Summary

- `for` and `while` are top-tested; `do-while` is bottom-tested.
- Only `do-while` guarantees at least one execution.
- Choice of loop depends on whether the iteration count is known and whether pre- or post-checking is required.

12. Programming Examples

The following solved programs apply the loop concepts covered above and are commonly asked in university and competitive examinations.

12.1 Sum of First N Natural Numbers

```
#include <stdio.h>
int main() {
    int i, n, sum = 0;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        sum += i;
    printf("Sum = %d", sum);
    return 0;
}
```

OUTPUT

```
Enter n: 5
Sum = 15
```

12.2 Factorial of a Number

```
#include <stdio.h>
int main() {
    int i, n;
    long fact = 1;
    printf("Enter n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
        fact *= i;
    printf("Factorial = %ld", fact);
    return 0;
}
```

OUTPUT

```
Enter n: 5
Factorial = 120
```

12.3 Multiplication Table

```c
#include <stdio.h>
int main() {
    int n = 7, i;
    for (i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", n, i, n * i);
    return 0;
}
```

OUTPUT

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

12.4 Fibonacci Series (using *while*)

```c
#include <stdio.h>
int main() {
    int a = 0, b = 1, c, count = 0, n = 8;
    while (count < n) {
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
        count++;
    }
    return 0;
}
```

OUTPUT

```
0 1 1 2 3 5 8 13
```

12.5 Prime Number Checking

```
#include <stdio.h>
int main() {
    int n = 29, i, isPrime = 1;
    for (i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            isPrime = 0;
            break;          // no need to check further
        }
    }
    if (isPrime)
        printf("%d is Prime", n);
    else
        printf("%d is Not Prime", n);
    return 0;
}
```

OUTPUT

```
29 is Prime
```

12.6 Pattern Printing (Number Pyramid)

```
#include <stdio.h>
int main() {
    int i, j, n = 4;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
    return 0;
}
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
```

12.7 Program Using *break* and *continue* — Sum Skipping Multiples of 3

```c
#include <stdio.h>
int main() {
    int i, sum = 0;
    for (i = 1; i <= 10; i++) {
        if (i % 3 == 0)
            continue;          // skip multiples of 3
        if (i == 9)
            break;             // stop entirely once i reaches 9
        sum += i;
    }
    printf("Sum = %d", sum);
    return 0;
}
```

OUTPUT

```
Sum = 22
/* i=1,2 added(3), 3 skipped, 4,5 added(12), 6 skipped, 7,8 added(22)+9 breaks */
```

12.8 Do-While Menu-Driven Example

```c
#include <stdio.h>
int main() {
    int choice;
    do {
        printf("1.Add 2.Subtract 3.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        // process choice using switch...
    } while (choice != 3);
    return 0;
}
```

NOTE: This is a classic real-world justification for `do-while` — the menu must display **at least once** before the exit condition can even be evaluated.

13. Quick Revision Section

13.1 Important Syntax Summary

Loop	Syntax
`for`	`for(init; condition; update) { body }`
`while`	`init; while(condition) { body; update; }`
`do-while`	`init; do { body; update; } while(condition);`
`break`	`break;` — exits innermost loop/switch
`continue`	`continue;` — skips to next iteration

13.2 Key Concepts at a Glance

- Loop = repeated execution of a block based on a condition
- Top-tested: `for`, `while` (0+ executions)
- Bottom-tested: `do-while` (1+ executions)
- Every loop needs Init → Condition → Update
- Nested loop: inner loop completes fully per outer pass
- Pre-op updates first; post-op uses old value first
- `break` exits the loop; `continue` skips an iteration
- `do-while` is the only loop needing a trailing `;`

13.3 Common Mistakes to Avoid

- Placing a stray semicolon after `for(...)` or `while(...)`, creating an empty-body loop.
- Forgetting to update the loop variable inside a `while` loop, causing an infinite loop.
- Forgetting the semicolon after `do { } while(condition)`.
- Confusing `break` (stops the loop) with `continue` (skips one iteration).
- Assuming `while` and `do-while` always behave the same — they differ when the condition is initially false.
- Mixing up pre-increment and post-increment results inside expressions.

13.4 Frequently Asked Exam Questions

1. Differentiate between entry-controlled and exit-controlled loops with examples.
2. Which loop guarantees at least one execution of its body, and why?
3. Write a program to check whether a number is prime using a `for` loop.
4. What is the difference between `break` and `continue`? Illustrate with an example.
5. Explain the working of a nested `for` loop with a pattern-printing example.
6. What happens if the update expression is omitted in a `for` loop?

7. Differentiate between pre-increment and post-increment operators with an example.
8. Compare `for`, `while`, and `do-while` loops on any three parameters.

13.5 One-Page Recap Card

Concept	Recap
Loop	Repeats a block while a condition holds true
3 loop types in C	`for`, `while`, `do-while`
Top-tested	Condition checked before body (`for`, `while`)
Bottom-tested	Condition checked after body (`do-while`)
Nested loop	Loop inside a loop; inner completes fully per outer iteration
`++i` / `i++`	Pre-increment updates first; post-increment uses old value first
`break`	Terminates the loop entirely
`continue`	Skips rest of current iteration only
Best for known count	`for` loop
Best for at-least-once	`do-while` loop

Final Exam Tip

When solving "predict the output" questions, always trace the loop step-by-step on paper: write the value of the control variable, evaluate the condition, note whether the body executes, then apply the update — repeat until the condition fails. This single habit eliminates most careless mistakes in loop-based questions.